Robotik Mühendisliği Serisi — Kitapçık 2

Yazılım: Linux, Git, C++, Python, ROS 2

Cilt 2'nin telefonda okunacak sürümü

Kodlar bu sürümde yok, kabuk komutları var (A4 Cilt 2 PDF'inde). Metin, sayısal örnekler, şekiller, 22 soru ve cevapları var.

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 Linux ve Kabuk **3**

1.1 Hayatta kalma komutları 3

1.2 Yönlendirme, boru, süreçler 3

1.3 Donanımla konuşmak 4

2 Git ve GitHub İş Akışı **5**

2.1 Temel döngü 5

2.2 Dallar ve çekme istekleri 5

2.3 İyi alışkanlıklar 6

3 C'den C++'a: RAII, Akıllı İşaretçiler, Taşıma **7**

4 Şablonlar, STL ve Lambda'lar **9**

5 Eşzamanlılık **10**

6 Gerçek Zamanlı Kontrol Döngüsü **11**

7 CMake, Proje Yapısı ve Testler **12**

7.1 Yapı 12

7.2 Derleme ve test 12

8 Robotik için Python **14**

8.1 Ortam 14

8.2 Alışkanlıklar ve tuzaklar 14

8.3 pytest 15

9 SB-1'in MuJoCo Modeli (MJCF) **16**

10 ROS 2 **18**

10.1 Temel komutlar 18

10.2 SB-1 IK düğümü 19

10.3 ros2_control ve hızlı döngü 19

11 Sorular **20**

12 Cevaplar **22**

Bu Cilt Hakkında

Diğer ciltler "ne hesaplanır" sorusunu cevaplıyor; Cilt 2 "bu hesaplar nasıl *güvenilir yazılıma* dönüşür" sorusunu. Bir robotik mühendisinin günlük işinin büyük kısmı burada: Linux terminali, Git, C++ ile gerçek zamanlı kod, derleme sistemi, test, Python ile analiz, simülasyon modeli ve ROS 2.

C biliyorsun; bu yüzden C++ bölümleri "C'de bunu nasıl yapardın, C++ neyi otomatikleştiriyor" karşılaştırmasıyla ilerliyor. Örneklerin hepsi seride tanıdık olan şeyler: SB-1'in bacak kinematiği, Cilt 4A'daki PID, servo okumaları.

Neler denendi, neler denenmedi: Bölüm 3–8'deki tüm C++ ve Python kodu bu belgede derlendi/çalıştırıldı, çıktılar gerçek. `sb1_core` projesi CMake ile kuruldu ve 7 GoogleTest testi geçti; Python sürümü pytest'te 27 test geçti. **ROS 2 kodu (Bölüm 10) ve MJCF modelinin MuJoCo'da yüklenmesi bu ortamda denenmedi** (ikisi de kurulu değil); dosyalar dikkatle yazıldı ama ilk çalıştırmada küçük düzeltmeler gerekebilir. Bunu açıkça işaretledim.

Bölüm	Konu	Kod
1	Linux ve kabuk	–
2	Git ve GitHub iş akışı	–
3	C'den C++'a: RAII, akıllı işaretçiler, taşıma	`y2_raii.cpp`
4	Şablonlar, STL, lambda	`y3_stl.cpp`
5	Eşzamanlılık: thread, mutex, atomic	`y4_threads.cpp`
6	Gerçek zamanlı kontrol döngüsü ve titreşim (jitter)	`y5_rtloop.cpp`
7	CMake, kütüphane yapısı, GoogleTest	`proje/sb1_core`
8	Robotik için Python: vektörleştirme, tuzaklar, pytest	`y7_python.py`, `proje/py_sb1`
9	SB-1'in MuJoCo modeli (MJCF) koddan	`y8_mjcf.py`
10	ROS 2: düğüm, konu, parametre, launch, ros2_control	`proje/ros2_sb1`
11–12	Sorular ve cevaplar	–

Staj dönemi için: telefon sürümü (Kitapçık 2) kodsuz ama kabuk komutları duruyor; 22 soru-cevap var. Bu cildi telefondan okurken her bölümün "kendine sor" kısmını not defterine yaz; bilgisayara kavuşunca ilk iş o komutları denemek.

Bölüm 1

Linux ve Kabuk

Robotların neredeyse tamamı Linux üzerinde çalışır (ROS 2'nin birincil platformu Ubuntu). Windows'ta WSL2 ile başlayabilirsin; ama gerçek zamanlı ve donanım işleri için doğrudan Ubuntu kurulumu daha az sorun çıkarır.

1.1 Hayatta kalma komutları

```
pwd; ls -la; cd ~/robot; cd ..          # neredeyim, ne var, git, geri
mkdir -p proje/src; cp -r a b; mv a b; rm -r eski   # dikkat: rm geri alınmaz
cat dosya; less uzun.log; head -n 20 x; tail -f log   # oku, canlı izle
grep -rn "kp" src/          # klasörde metin ara (satır numarasıyla)
find . -name "*.cpp"         # dosya ara
chmod +x betik.sh; ./betik.sh # çalıştırılabilir yap, çalıştır
sudo apt install build-essential cmake git   # paket kur
```

1.2 Yönlendirme, boru, süreçler

```
./robot_log > log.txt 2>&1          # stdout ve stderr dosyaya
python3 analiz.py < veri.csv | sort | uniq -c   # boru (pipe): çıktı -> girdi
ps aux | grep controller; kill -INT 1234     # süreci bul, Ctrl+C eşdeğeri
gönder
htop                                  # CPU/bellek canlı
export ROS_DOMAIN_ID =7; echo $PATH   # ortam değişkenleri
```

1.3 Donanımla konuşmak

```
ls /dev/ttyUSB* /dev/ttyACM*        # Arduino genelde /dev/ttyACM0
sudo usermod -aG dialout $USER       # seri porta sudo'suz erişim
(çıkış-giriş yap)
dmesg | tail                          # USB takınca çekirdek ne dedi?
screen /dev/ttyACM0 115200            # seri terminal (çıkış: Ctrl+A, K)
ssh pi@192.168.1.20; scp derleme.tar pi@192.168.1.20:~   # robottaki
bilgisayara bağlan, dosya at
```

Kendine sor

Bir program "Permission denied" diyorsa üç olasılık: dosya çalıştırılabilir değil (`chmod +x`), cihaz dosyasına erişim yok (`dialout` grubu), ya da gerçekten yönetici yetkisi gerek (`sudo`; ama ROS düğümlerini asla `sudo` ile çalıştırma).

Bölüm 2

Git ve GitHub İş Akışı

Git, kodun zaman makinesidir: her *commit* projenin bir anlık görüntüsü. Lisansüstü başvurularında GitHub profilin, CV'nin yazılı olmayan yarısıdır.

2.1 Temel döngü

```
git init; git clone https://github.com/kullanici/sb1.git
git status                    # ne değişti?
git diff                      # satır satır ne değişti?
git add src/leg.cpp           # sahneye al (sadece istediğin dosyalar)
git commit -m "leg: IK erişim dışı durumu nullopt döndürüyor"
git log --oneline --graph     # geçmiş
git push origin main          # GitHub'a gönder
```

2.2 Dallar ve çekme istekleri

```
git switch -c ozellik/dcm-planner     # yeni dal
# ... değişiklikler, commit'ler ...
git push -u origin ozellik/dcm-planner
# GitHub'da Pull Request aç, CI testleri çalışsın, incele, birleştir
git switch main; git pull             # birleştirilmiş main'i al
git rebase main                       # (dal eskidiyse) dalını main'in
üstüne taşı
```

2.3 İyi alışkanlıklar

- **Küçük, anlamlı commit'ler.** Mesaj: "ne" ve "neden". "düzeltme" kötü; "pid: türevi ölçümden al (basamakta tekme vardı)" iyi.
- `.gitignore`: `build/`, `__pycache__/`, `*.o`, büyük veri ve videolar depoya girmez.
- **Asla** şifre, API anahtarı veya kişisel veri commit'leme; commit'lendiyse geçmişten silmek zordur, anahtarı iptal et.
- Her depoya bir `README.md`: ne yapar, nasıl derlenir, bir görsel/video, sonuçlar. Bir hocanın 30 saniyede anlayabileceği kadar.
- GitHub Actions ile her push'ta testleri çalıştır (Bölüm 7'deki `ctest`).

En sık felaket

`git reset --hard` ve `git push --force` geri alınamayan iş kaybı yaratabilir. Emin değilsen önce `git stash` veya yeni bir dal aç. Kaybolan commit'ler çoğu zaman `git reflog` ile bulunur.

Bölüm 3

C'den C++'a: RAII, Akıllı İşaretçiler, Taşıma

C'de her `malloc`'un bir `free`'si, her `fopen`'ın bir `fclose`'u olmalı ve fonksiyonun *her* çıkış yolunda (erken `return`, hata) bunu hatırlaman gerekir. C++'ın en önemli fikri bunu derleyiciye devretmek:

RAII (Resource Acquisition Is Initialization): kaynağı kurucu (constructor) alır, yıkıcı (destructor) bırakır. Nesne kapsamdan çıktığı an, nasıl çıkarsa çıksın, yıkıcı çağrılır. Seri port, dosya, mutex kilidi, GPU belleği, ROS düğümü: hepsi böyle yönetilir.

- `std::unique_ptr<T>`: tek sahip; kopyalanamaz, `std::move` ile devredilir. Varsayılan seçim.
- `std::shared_ptr<T>`: paylaşılan sahiplik, referans sayacı; son sahip bırakınca silinir. ROS 2 mesajları ve düğümleri böyle dolaşır. Bedeli: sayaç (atomik işlem).
- Ham işaretçi (`T*`) ve referans (`T&`): "sahip olmadan bakmak" için. `new`/`delete`'i elle yazman neredeyse hiç gerekmez.
- **Taşıma**: `T(T&&)` kurucusu kaynağı kopyalamak yerine "çalar" ve kaynağı boş bırakır. Büyük vektörler ve kopyalanamayan kaynaklar (port) böyle verimli dolaşır.
- **Beşin kuralı**: kaynağı elle yöneten bir sınıf yazıyorsan yıkıcı, kopya kurucu/atama, taşıma kurucu/atama beşini birlikte düşün. Çoğu zaman sıfır kuralı daha iyi: üyeleri zaten RAII olan türlerden (vector, unique_ptr) kur, hiçbirini yazma.

Çıktıda dikkat: erken `return`'de de port kapandı; `unique_ptr` kapsam

sonunda servoyu sildi; `shared_ptr` sayacı 2'den 1'e düştü ve `reset` ile silindi; vektördeki portlar `main` bitince kapandı ve hiçbiri iki kez kapanmadı (taşınan nesnenin `name_`'i `nullptr` yapıldı).

C ile karşılaştırma

C'de `struct` + `port_open()`/`port_close()` fonksiyonları yazardın ve `goto cleanup` kalıbı kullanırdın. C++'ta `class` aynı `struct`'tır (varsayılan erişimi `private`); fark, kurucu/yıkıcının dil tarafından garanti edilmesi. `= delete` bir fonksiyonu yasaklar: kopyalamaya çalışan kod derlenmez, hata çalışma zamanına kalmaz.

Bölüm 4

Şablonlar, STL ve Lambda'lar

Şablon (template): türden bağımsız kod. `template <typename T, std::size_t N> class RingBuffer` bir kez yazılır; `RingBuffer<double,5>` ve `RingBuffer<int,100>` için derleyici ayrı, tam hızda kod üretir. Eigen'in `Matrix<double,3,3>`'ü (Cilt 1, 5) de böyle çalışır.

STL kapları: `std::vector` (dinamik dizi, varsayılan seçim), `std::array` (sabit boyut, yığında), `std::map`/`std::unordered_map` (anahtar–değer), `std::deque`, `std::queue`. **Algoritmalar:** `sort`, `nth_element`, `count_if`, `accumulate`, `find_if`, `transform`. **Lambda:** yerinde tanımlanan isimsiz fonksiyon, `[yakalama](parametreler){gövde}`; `[&]` çevredeki değişkenleri referansla, `[ =]` kopyayla yakalar.

Medyan filtre dersi: 3.50 ve −2.00 sıçramaları ortalamayı 1.50 ve 0.40'a çekti; medyan her zaman 1.00 civarında kaldı. Potansiyometre ve kızılötesi mesafe sensörlerinde tek örneklik bozulmalara karşı ucuz ve etkili (bedeli: biraz gecikme; Cilt 3 Bölüm 4).

Gerçek zamanlı kodda STL

`std::vector::push_back`, `std::map` ekleme ve `std::string` birleştirme *bellek ayırabilir*; 1 kHz döngüde bu, öngörülemeyen gecikme demektir. Kuralı: döngüden önce `reserve` et veya sabit boyutlu yapılar (`std::array`, halka tampon) kullan; döngü içinde `new`, `malloc`, dosya/konsol G/Ç yapma.

Bölüm 5

Eşzamanlılık

Bir robotta aynı anda birkaç iş yürür: sensör okuma, durum kestirimi, kontrol, haberleşme, kayıt. Bunlar ayrı **iş parçacıklarında** (thread) çalışır ve veri paylaşır. Paylaşılan veriye iki iş parçacığı aynı anda, en az biri yazarak erişirse **veri yarışı** (data race) olur: C++'ta bu tanımsız davranıştır; program "çoğu zaman" çalışıp bir gün robotu düşürür.

Araçlar:

- `std::mutex` + `std::lock_guard` (RAII kilit): kritik bölgeye tek iş parçacığı.
- `std::atomic<T>`: tek bir değişken için kilitsiz, bölünmez işlemler (bayraklar, sayaçlar).
- `std::condition_variable`: "veri gelene kadar uyu" (üretici–tüketici).
- Gerçek zamanlı döngüde kilit bile risklidir (düşük öncelikli iş parçacığı kilidi tutarken yüksek öncelikli bekler: *öncelik terslemesi*). Çözümler: kilitsiz tek-yazar/tek-okur kuyruklar, çift tampon; `ros2_control`'ün `realtime_tools` paketi bu kalıpları hazır verir.

Korumasız sayaçta 4004 artırmadan sadece 2002'si kaldı (bu koşuda; sonuç her çalıştırmada değişebilir, ki sorun da tam olarak bu). Mutex ve atomic her seferinde doğru 400 000'i verdi. Teşhis için: `g++ -fsanitize=thread` (ThreadSanitizer) yarışı çalışırken yakalar.

Bölüm 6

Gerçek Zamanlı Kontrol Döngüsü

Cilt 4A Bölüm 3: gecikme ve titreşim (jitter) faz payını yer. Bir kontrol döngüsünün iki özelliği önemli: **ortalama periyot** doğru mu ve **en kötü durum** (max, p99) ne kadar kötü. "Gerçek zamanlı" hızlı demek değil, *öngörülebilir* demektir.

İki döngü kalıbı:

- `work(); sleep_for(1ms);` — periyot = iş süresi + 1 ms + uyanma gecikmesi: **kayar**.
- `next += 1ms; sleep_until(next);` — mutlak zaman çizelgesine göre: hatalar birikmez.

`sleep_for` ile 1000 döngü 1389 ms sürdü (%39 yavaş: 0.18 ms iş + uyanma gecikmeleri); `sleep_until` ile 1000.1 ms. Ama `sleep_until`'de bile en kötü periyot 1.85 ms: sıradan bir Linux (hele sanal bir bulut makinesi; bu ölçümler öyle bir makinede alındı) zaman garantisi vermez.

Gerçek robotta: `PREEMPT_RT` çekirdek yaması (Ubuntu'nun gerçek zamanlı çekirdeği), iş parçacığını `SCHED_FIFO` ve yüksek öncelikle çalıştırmak, belleği kilitlemek (`mlockall`), bir CPU çekirdeğini döngüye ayırmak (`isolcpus`, `taskset`), döngüde bellek ayırma/G/Ç yapmamak. Böylece p99 titreşim tipik olarak onlarca mikrosaniyeye iner. SB-1'de hobi servoları zaten 50 Hz komut alıyor; orada en hassas döngü Arduino'daki (Cilt 3), PC tarafında 200–500 Hz yeterli.

Bölüm 7

CMake, Proje Yapısı ve Testler

Tek dosyalık `g++ a.cpp` bir yere kadar; gerçek proje kütüphaneler, uygulamalar ve testlerden oluşur. **CMake** bunları platformdan bağımsız tarif eder; ROS 2'nin `colcon`'u da altta CMake kullanır.

7.1 Yapı

```
sb1_core/
  CMakeLists.txt
  include/sb1/leg.hpp  pid.hpp      # genel arayüz (başka projeler bunu
  görür)
  src/leg.cpp  pid.cpp              # gerçekleme
  apps/leg_demo.cpp                 # çalıştırılabilir örnek
  tests/test_leg.cpp  test_pid.cpp  # GoogleTest
```

Başlık (`.hpp`) ne yapıldığını, kaynak (`.cpp`) nasıl yapıldığını söyler. `namespace sb1` isim çakışmasını önler. `#pragma once` başlığın iki kez dahil edilmesini engeller.

7.2 Derleme ve test

```
cmake -S . -B build -DCMAKE_BUILD_TYPE =Release
cmake --build build -j4
./build/leg_demo
cd build && ctest --output-on-failure
```

Test gerçekten işe yarıyor mu? Kasıtlı bir hata ekledim: `pid.cpp`'de anti-windup koşulunu silip integrali her zaman güncelledim. Derleyici `-Wall -Wextra` sayesinde "kullanılmayan değişken: saturating" uyarısı verdi ve test düştü: Buna *mutasyon testi* denir: testlerin bir hatayı yakalayabildiğini ancak hatayı bilerek sokunca anlarsın. İlk yazdığımda bu testi bir hatayla (eski derlemeyi test ederek) "geçti" sandım; temiz bir `build` klasörüyle yeniden yapınca düştü. Ders: şüphelendiğinde `rm -rf build`.

Test edilebilir robot kodu

Donanıma dokunan kodu (seri port, servo) ince bir katmanda tut; hesap yapan kodu (IK, PID, kestirici) saf fonksiyonlar/sınıflar olarak ayır. Böylece hesap kodunu robotsuz, her commit'te, saniyeler içinde test edersin. Cilt 3'teki `arduino_mock.hpp` bunun küçük bir örneğiydi.

Bölüm 8

Robotik için Python

Python; analiz, prototip, simülasyon betikleri, veri işleme ve öğrenme (Cilt 6 Bölüm 5) için. C++ ise robotun üstünde, hızlı döngülerde. İkisini `pybind11` ile bağlamak yaygın: C++ kütüphanesi (`sb1_core`) Python'dan çağrılır.

8.1 Ortam

```
python3 -m venv .venv && source .venv/bin/activate   # projeye özel ortam
pip install numpy scipy matplotlib pytest mujoco
pip freeze > requirements.txt                          # tekrarlanabilirlik
```

8.2 Alışkanlıklar ve tuzaklar

- **Vektörleştir:** NumPy işlemleri dizinin tamamında C hızında çalışır; Python `for` döngüsü her eleman için yorumlayıcı maliyeti öder.
- **`@dataclass`** parametre kümeleri için; `frozen =True` kazara değişikliği engeller. Tip ipuçları (`def f(q: np.ndarray) -> float`) okunabilirlik ve `mypy` denetimi için.
- `if __name__ == "__main__":` ile betiği hem modül hem program yap.

100 000 ileri kinematik hesabında vektörleştirme yaklaşık 30 kat hızlı (bu makinede 372 ms'ye karşı 12 ms).

8.3 pytest

`parametrize` ile $5 \times 5 = 25$ eklem açısı çiftinde “IK(FK(q)) eşittir q” kontrol edildi; artı 2 test, toplam 27.

Bölüm 9

SB-1'in MuJoCo Modeli (MJCF)

MuJoCo modeli bir XML dosyasıdır: gövdeler (`body`) iç içe ağaç oluşturur; her gövdenin ataleti (`inertial`), ebeveynine göre eklemi (`joint`), çarpışma geometrisi (`geom`) vardır; `actuator` eklemleri sürer, `sensor` ölçer. Menagerie'deki `robotis_op3` ve `berkeley_humanoid` modellerini açıp okumak en iyi öğretmen.

Bu bölümde XML'i elle değil, **Python'dan üretiyoruz**: kütle ve boyutlar tek yerde, ataletler Cilt 1'in kutu formülüyle hesaplanıyor, iki bacak aynı fonksiyonla simetrik kuruluyor. Parametreler Cilt 6 Bölüm 4'ün sonucuna göre **hafif** seçildi (toplam 1.28 kg): böylece MG996R diz, tek ayakta diz–ayak yatay mesafesi 8.6 cm'ye kadar (kabaca 50° diz bükümüne kadar) ağırlığı taşıyabiliyor; ama bu durma torku, sürekli çalışma için daha temkinli olmak gerekir.

Denenmedi

MuJoCo bu ortamda kurulu olmadığından model burada yüklenip simüle edilmedi; yalnızca XML'in geçerli olduğu, 10 eklem/aktüatör ve kütle toplamı kontrol edildi. Kendi bilgisayarında `python -m mujoco.viewer --mjcf =sb1.xml` ile aç; ilk kontrol listesi: robot yere doğru yükseklikte mi duruyor, eklem eksenleri ve sınırları doğru yönde mi, ayaklar yere giriyor mu, `kp` ile servo davranışı Cilt 3'teki ölçümlere benziyor mu.

Notlar: `position` aktüatörü MuJoCo'da bir PD konum servosu modelidir ($\tau = k_p(q_{\text{hedef}} - q)$, `forcerange` ile kırpılı); hobi servosunun iç denetleyicisine kabaca benzer. Cilt 3'teki gecikme ve hız sınırını eklemek

için kontrol döngüsünde komutu 20 ms geciktir ve eğim sınırla. `armature` motorun yansıyan rotor ataletidir ($J_m N^2$; Cilt 3'teki $J_m = 2 \cdot 10^{-7}$, $N = 195$ ile ≈ 0.008; buradaki 0.002 kaba bir başlangıç değeri, sistem tanılama ile ayarlanmalı).

Bölüm 10

ROS 2

ROS 2, robot yazılımını birbirinden bağımsız **düğümlere** (node) böler; düğümler **konular** (topic, yayın/abone), **servisler** (istek/cevap) ve **eylemler** (uzun süren, geri bildirimli görevler) ile konuşur. Altta DDS ara katmanı çalışır. Kurulum: Ubuntu 24.04 + ROS 2 Jazzy (veya 22.04 + Humble).

Denenmedi

Bu bölümdeki kod ROS 2 kurulu olmadığı için burada derlenmedi; rclcpp'nin Humble/Jazzy API'sine göre yazıldı. Küçük derleme hataları çıkarsa hata mesajını oku: genelde eksik bir `find_package` veya başlık yolu.

10.1 Temel komutlar

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker          # bir terminalde
ros2 topic list; ros2 topic echo /chatter; ros2 topic hz /chatter
ros2 node info /talker; ros2 param list
mkdir -p ~/ws/src && cd ~/ws && colcon build --symlink-install && source
install/setup.bash
ros2 launch sb1_ros leg.launch.py
ros2 topic pub /foot_target geometry_msgs/msg/Point "{x: 0.03, y: 0.0, z:
-0.35}"
```

10.2 SB-1 IK düğümü

Dikkat edilecekler: düğüm sınıfı `rclcpp::Node`'dan türer; yayıncı, abone ve zamanlayıcı `shared_ptr` olarak üyelerde tutulur (yoksa kapsam bitince yok olur: Bölüm 3); geri çağırmalar lambda; parametreler `declare_parameter` ile; hesap kodu (`sb1::inverse`) ROS'tan bağımsız kütüphanede, böylece Bölüm 7'deki testler ROS'suz çalışır.

10.3 ros2_control ve hızlı döngü

Konu tabanlı haberleşme 10–100 Hz için rahat; 1 kHz eklem kontrolü için değil. `ros2_control` donanımı bir *donanım arayüzü* (`read()`/`write()`) ile soyutlar, denetleyiciler (ör. `joint_trajectory_controller`, kendi yürüme denetleyicin) aynı gerçek zamanlı döngüde, mesajlaşma olmadan çalışır. SB-1 için yol: Arduino ile seri haberleşen bir `SystemInterface` eklentisi, üstünde Cilt 6'daki DCM/WBC'yi içeren bir denetleyici. Simülasyonda aynı denetleyiciyi MuJoCo'ya bağlayan köprüler mevcut; böylece simülasyon ve gerçek robot aynı kodu çalıştırır.

Bölüm 11

Sorular

(K): kod okuma; kâğıtta çıktıyı tahmin et.

Y1–2 — Linux ve Git

Y1.1 `./robot > log.txt 2>&1` ne yapar? `2>&1` olmasaydı hata mesajları nereye giderdi?

Y1.2 Arduino'yu taktın, `/dev/ttyACM0` açılırken "Permission denied". İki doğru çözüm ve bir yanlış çözüm söyle.

Y1.3 `grep -rn "Kp" src/ | wc -l` ne üretir?

Y2.1 `git add` ile `git commit` arasındaki fark nedir? Neden iki adım?

Y2.2 Bir PID değişikliği için kötü ve iyi bir commit mesajı yaz.

Y2.3 `build/` klasörü neden depoya girmemeli?

Y3–4 — C++

Y3.1 RAII'yi bir cümleyle tanımla. C'deki `goto cleanup` kalıbıyla ilişkisi?

Y3.2 (K) `auto s = std::make_unique<Servo>(1); auto t = std::move(s);` sonrası `s` ne durumda? `s->id` yazarsan ne olur?

Y3.3 (K) `shared_ptr a`; içte `b = a`; iç kapsam bitince ve `a.reset()` sonrası sayaçlar ve silinme anı?

Y3.4 `SerialPort(const SerialPort&) = delete;` neden? Yazılmasaydı `std::vector<SerialPort>` kopyalandığında ne olurdu?

Y4.1 (K) Halka tampon $N = 5$, sırayla 1, 1, 1, 9, 1 eklendi. Ortalama ve medyan?

Y4.2 (K) `std::count_if(v.begin(), v.end(), [](int x){return x > 2;})`, `v ={1,5,2,7,3}`?

Y4.3 Gerçek zamanlı döngü içinde neden `push_back` ve `printf` sakıncalı?

Y5–6 — Eşzamanlılık ve gerçek zaman

Y5.1 Veri yarışı nedir? Korumasız sayaç neden 4004 yerine 2002 verebilir (bir senaryo yaz)?

Y5.2 Bir `bool estop` bayrağı için mutex mi atomic mi?

Y6.1 İş 0.18 ms, `sleep_for(1ms)` ve uyanma gecikmesi ortalama 0.2 ms. 1000 döngü kaç ms sürer? `sleep_until` ile?

Y6.2 "Gerçek zamanlı = hızlı" ifadesi neden yanlış?

Y7–10 — Proje, Python, MJCF, ROS 2

Y7.1 Başlık ve kaynak dosyası ayrımı neden? `target_include_directories(... PUBLIC include)`'daki `PUBLIC` ne demek?

Y7.2 Mutasyon testi nedir; bu ciltte neyi gösterdi?

Y8.1 (K) `def f(x, l =[]): l.append(x); return l. f(1); print(f(2))` ne yazdırır? Düzelt.

Y9.1 Uyluk 0.15 kg, $0.04 \times 0.04 \times 0.20$ m kutu: kütle merkezinden geçen yatay eksene göre atalet? Kalça eksenine (uç) göre?

Y10.1 Konu, servis ve eylem arasındaki fark; SB-1'den birer örnek.

Bölüm 12

Cevaplar

Y1–2

Y1.1 Standart çıktıyı `log.txt`'ye yazar, standart hatayı (2) da standart çıktının (1) gittiği yere yönlendirir. Olmasaydı hatalar terminale düşerdi, dosyada olmazdı.

Y1.2 Doğru: kullanıcıyı `dialout` grubuna ekleyip yeniden giriş yapmak; bir udev kuralıyla cihaza izin vermek. Yanlış: programı `sudo` ile çalıştırmak (veya `chmod 777 /dev/ttyACM0`, her takışta sıfırlanır).

Y1.3 `src/` altında "Kp" geçen satır sayısı.

Y2.1 `add` değişikliği bir sonraki anlık görüntüye (sahne) koyar; `commit` sahnedekini kaydeder. İki adım, ilgisiz değişiklikleri ayrı commit'lere bölmeni sağlar.

Y2.2 Kötü: "düzeltme". İyi: "pid: türevi ölçümden al; hedef basamağında 400 N · m tekme vardı (Cilt 4A E2.4)".

Y2.3 Makineye özgü, yeniden üretilebilir, büyük ve sürekli değişen dosyalar; depoyu şişirir, çakışma üretir.

Y3–4

Y3.1 Kaynağın ömrü nesnenin ömrüne bağlıdır: kurucu alır, yıkıcı bırakır. `goto cleanup`'ın elle yaptığını (her çıkışta temizlik) derleyici garanti eder.

Y3.2 `s` boş (`nullptr`); `s->id` tanımsız davranış, büyük olasılıkla çökme.

Y3.3 İçeride 2, iç kapsam bitince 1, `reset` ile 0 ve o anda silinir.

Y3.4 İki nesne aynı portu temsil eder ve ikisinin yıkıcısı da kapatmaya çalışır (çift kapatma). `= delete` ile kopyalayan kod derlenmez.

Y4.1 Ortalama $13/5 = 2.6$; sıralı $1, 1, 1, 1, 9 \Rightarrow$ medyan 1.

Y4.2 5, 7, 3: **3**.

Y4.3 `push_back` kapasite dolunca bellek ayırır (süresi belirsiz, kilit alabilir); `printf` G/Ç yapar ve bloklayabilir. İkisi de döngünün en kötü süresini öngörülemez yapar.

Y5–6

Y5.1 İki iş parçacığı aynı veriye eşzamanlı, en az biri yazarak, senkronizasyonsuz erişir. Senaryo: A `t =racy` (0) okur, B `t =racy` (0) okur, A 1 yazar, B 1 yazar: iki artırma, sonuç 1. Bu her adımda olursa yarısı kaybolur.

Y5.2 `std::atomic<bool>`: tek değişken, kilitsiz, gerçek zamanlı döngüyü bekletmez.

Y6.1 $1000 \cdot (0.18 + 1 + 0.2) = 1380$ ms (ölçülen 1389). `sleep_until`: ≈ 1000 ms (iş + gecikme 1 ms'nin altında kaldıkça).

Y6.2 Gerçek zamanlılık, *son tarihin her seferinde* karşılanması (öngörülebilirlik, en kötü durum) demektir; ortalaması hızlı ama arada bir 5 ms takılan bir döngü gerçek zamanlı değildir.

Y7–10

Y7.1 Başlık arayüzü (ne), kaynak gerçeklemeyi (nasıl) taşır: kullanıcılar sadece başlığı görür, değişiklikte sadece ilgili kaynak yeniden derlenir. `PUBLIC`: bu dizin hem kütüphanenin kendisine hem de ona bağlanan hedeflere (testler, uygulama) aktarılır.

Y7.2 Koda bilerek hata sokup testlerin onu yakalayıp yakalamadığını görmek. Anti-windup silinince `NoWindupWhileSaturated` düştü; derleyici

uyarısı da ipucu verdi.

Y8.1 `[1, 2]`: varsayılan liste bir kez oluşturulur, çağrılar arasında paylaşılır. Düzeltme: `def f(x, l =None): l = [] if l is None else l; ...`

Y9.1 $I_{\text{cm}} = \frac{m}{12}(b^2+c^2) = \frac{0.15}{12}(0.0016+0.04) = 5.2\cdot10^{-4}$ kg·m^2. Uca göre (paralel eksen): $5.2\cdot10^{-4} + 0.15\cdot0.1^2 = 2.02\cdot10^{-3}$.

Y10.1 Konu: sürekli veri akışı, yayın/abone (`/imu`, `/joint_states`). Servis: kısa istek–cevap (“kalibrasyon sıfırını kaydet”). Eylem: uzun süren, geri bildirimli, iptal edilebilir görev (“şu noktaya yürü”).